

Proyecto HaDes-V

Informe sobre la participación en el *Community Challenge with HaDes-V* de RISC-V International

Julián Hinojosa Gil

Estudiante Ingeniería Informática | Universidad de Alicante

Versión 1.1 – 28 de julio de 2026

Resumen

Este documento presenta la memoria técnica del desarrollo, optimización y validación física del procesador **HaDes-V**. Partiendo de la especificación para implementar un núcleo docente escalar de 32 bits conforme a la especificación libre **RISC-V (RV32I)** con segmentación de 5 etapas, se diseñó e integró un sistema de especulación de control compuesto por un predictor de saltos dinámico bimodal (*Branch History Table* - BHT) de 4096 entradas con contadores saturantes de 2 bits y un acelerador de saltos incondicionales (*JAL Bypass*). Posteriormente, se resolvió el «Muro de la Memoria» mediante una jerarquía L1 en **Arquitectura Harvard Modificada**, incorporando una caché de instrucciones de 4 KB (I-Cache), una caché de datos de 4 KB (D-Cache), un *Write Buffer* asíncrono con política *Fire & Forget* y una línea de invalidación por hardware para mantener la coherencia de código mediante la instrucción **FENCE.I**. La telemetría en hardware mediante contadores de rendimiento (HPCs) en la etapa de *Writeback* permitió validar un incremento de rendimiento del 63,8 % (elevando el IPC de 0,506 a 0,829) y eliminar hasta un 85,6 % de las paradas de memoria. El procesador fue sintetizado en la FPGA **Digilent Nexys A7-100T** (Xilinx Artix-7) alcanzando un cierre de tiempos positivo ($WNS = +0,036$ ns) a una frecuencia de reloj de **100 MHz**.



Figura 1: [Insignia de oro](#) tras conseguir implementar y validar el diseño completo en la FPGA.

Índice

1. Resumen Ejecutivo e Introducción	3
1.1. Presentación del Proyecto HaDes-V	3
1.2. Motivaciones para la mejora del diseño base: Los Muros del Control y de la Memoria	3
1.3. Resumen de Contribuciones y Resultados Clave	4
1.4. Estructura del Informe	4
2. Contexto y Marco del Proyecto	4
2.1. El Community Challenge with HaDes-V	4
2.2. Arquitectura de Partida	4
2.3. Entorno de Desarrollo y Flujo de Trabajo Hardware/Software	5
3. Diseño Microarquitectónico e Innovaciones	5
3.1. Telemetría en Hardware: Performance Counters (HPCs)	5
3.1.1. Motivación y Necesidad de Observabilidad	5
3.1.2. Interfaz de Software mediante CSRs y Control del Efecto Observador	6
3.2. Especulación de Control (BHT y JAL Bypass)	6
3.2.1. Predictor Dinámico Bimodal mediante BHT	6
3.2.2. Aceleración Transparente mediante JAL Bypass	7
3.3. Jerarquía de Memoria L1 Harvard Modificada y Coherencia	7
3.3.1. Justificación Arquitectónica: De Von Neumann a Harvard Modificada	7
3.3.2. Estructura del Subsistema L1	8
3.3.3. Write Buffer no bloqueante y Enrutamiento MMIO	8
3.3.4. Coherencia de Instrucciones mediante FENCE.I	9
4. Diagnóstico y Resolución de Fallos	9
4.1. Filosofía del Análisis Post-Mortem	9
4.2. Depuración del Control e I-Cache	9
4.3. Depuración de la D-Cache, Write Buffer y Periféricos MMIO (Canal de Datos)	10
5. Evaluación Cuantitativa de Rendimiento y Cierre en FPGA	11
5.1. Metodología de Evaluación y Cargas de Trabajo	11
5.2. Análisis del Instruction Mix y Desglose del Cauce de Instrucciones	11
5.3. Evolución Microarquitectónica: Baseline vs. BHT vs. L1 Final	12
5.3.1. Validación del Determinismo Matemático en la BHT	12
5.3.2. Análisis de Aliasing y Capacidad: BHT 1K vs. BHT 4K	12
5.3.3. Eficiencia del Subsistema L1 y Absorción de Latencias	13
5.4. Síntesis Física, Recursos y Cierre de Tiempos en Vivado	14
5.4.1. La Importancia del Cierre Temporal en Ingeniería de Semiconductores	14
5.4.2. Análisis del Cierre Temporal y Evolución del Camino Crítico	14
6. Conclusiones y Trabajo Futuro	15
6.1. Síntesis de Logros y Cumplimiento de Objetivos	15
6.2. Lecciones de Ingeniería RTL	15
6.3. Líneas de Trabajo Futuro	15
7. Referencias	16

1 Resumen Ejecutivo e Introducción

1.1 Presentación del Proyecto HaDes-V

El proyecto **HaDes-V** documenta la implementación de un diseño de procesador docente [2] en un microcontrolador completamente funcional y desplegado en hardware real.

Para comprender el punto de partida sin perderse en tecnicismos y siglas, es importante definir las bases de la arquitectura:

- **RISC-V (RV32I):** Arquitectura de conjunto de instrucciones (*ISA*) abierta y libre de 32 bits [3]. La especificación RV32I constituye el subconjunto base de instrucciones enteras (operaciones aritmético-lógicas, accesos a memoria y saltos de control), actuando como el «vocabulario estándar» que entiende el procesador.
- **Procesador Escalar:** Modelo microarquitectónico donde el cauce de instrucciones emite y retira como máximo una única instrucción por ciclo de reloj ($IPC_{ideal} = 1,0$), procesando operandos sobre un único canal de datos (a diferencia de arquitecturas superescalares de múltiple emisión o procesadores vectoriales).
- **Ejecución Bare-Metal:** Paradigma de funcionamiento en el cual el programa ejecutable interactúa directamente sobre la electrónica y los registros del hardware, sin capas de abstracción ni la intermediación de un sistema operativo.
- **Modo Máquina (M-Mode):** Nivel de privilegio de ejecución único implementado en el procesador, que otorga acceso directo, completo e irrestricto a todos los recursos físicos del silicio y a la gestión global del sistema.
- **Bus Wishbone:** Estándar abierto e independiente de arquitectura para la interconexión síncrona en sistemas en chip (*SoC*) [4]. Define la interfaz de señales mediante la cual la CPU se comunica con la jerarquía L1, la memoria RAM y los periféricos.
- **MMIO (*Memory-Mapped I/O*):** Esquema de entrada/salida donde los registros de control de los periféricos (UART, LEDs, contadores de rendimiento) se ubican dentro del mismo espacio de direccionamiento que la RAM, permitiendo su acceso mediante instrucciones estándar de lectura y escritura (LW/SW).
- **Interfaz Serie UART (*Universal Asynchronous Receiver-Transmitter*):** Protocolo de comunicación serie asíncrono empleado por el *Bootloader* para recibir los programas ejecutables desde el ordenador anfitrión y cargarlos dinámicamente en la memoria del microcontrolador.
- **Registros CSR (*Control and Status Registers*):** Registros del sistema pertenecientes a la especificación RISC-V (independientes del banco de registros general $x0-x31$) destinados a la configuración, consulta del estado interno del procesador y extracción de telemetría de rendimiento en hardware.
- **Síntesis en FPGA y BRAM:** Proceso de síntesis física e implementación del diseño RTL (escrito en SystemVerilog) sobre un circuito integrado reconfigurable (*FPGA* Xilinx Artix-7), empleando bloques de memoria SRAM interna (*Block RAM* – BRAM) para albergar la memoria principal del microcontrolador.

1.2 Motivaciones para la mejora del diseño base: Los Muros del Control y de la Memoria

Idea Clave: El Límite del Rendimiento en Procesadores Escalares

En un procesador segmentado ideal, el objetivo es completar una instrucción por ciclo de reloj ($CPI_{ideal} = 1,0$). Sin embargo, el rendimiento real está limitado por la suma de penalizaciones [1]:

$$CPI_{real} = CPI_{ideal} + CPI_{control} + CPI_{memoria}$$

- $CPI_{control}$ (**Muro del Control**): Ciclos desperdiciados vaciando la tubería (*flushes*) tras calcular tarde la dirección de un salto.
- $CPI_{memoria}$ (**Muro de la Memoria**): Ciclos en los que la CPU se congela (*stalls*) esperando la respuesta de la RAM principal.

En un procesador segmentado clásico de 5 etapas (*Fetch, Decode, Execute, Memory, Writeback*), la ejecución avanza como una cadena de montaje. Sin embargo, en el diseño base de HaDes-V surgían dos cuellos de botella fundamentales:

1. **El Muro del Control (Saltos Condicionales):** La dirección a la que debe saltar un bucle (como un `for` o un `if`) no se calcula hasta la 3.^a etapa (*Execute*). Sin un mecanismo de predicción, la CPU asumía de forma estática que el salto nunca se tomaría (*Predict Not-Taken*). Si la suposición fallaba, el procesador debía vaciar o purgar (*flush*) las dos instrucciones cargadas por error en la tubería, tirando a la basura **2 ciclos de reloj** por cada salto.
2. **El Muro de la Memoria (Latencia de RAM):** Las instrucciones de lectura (`LW`) y escritura (`SW`) deben comunicarse con la memoria RAM principal a través del bus del sistema. Mientras la memoria principal responde, la CPU debe congelar la segmentación (*stall*), perdiendo valiosos ciclos en los que no se realiza cómputo útil.

1.3 Resumen de Contribuciones y Resultados Clave

Para superar ambos muros, en este trabajo se diseñaron e implementaron sobre el diseño base tres grandes bloques microarquitectónicos:

1. **Telemetría en Hardware (HPCs):** Incorporación de un conjunto de contadores de rendimiento de hardware (*Hardware Performance Counters*) mapeados en registros especiales de control (*CSRs*) en la etapa final de *Writeback*, permitiendo medir ciclos, instrucciones, paradas y purgas sin alterar el camino crítico.
2. **Especulación de Control (BHT y JAL Bypass):** Creación de una tabla de historial de saltos (*Branch History Table* - BHT) de 4096 entradas con contadores saturantes de 2 bits que predice la dirección de los bucles en la etapa de *Fetch*, junto con un decodificador rápido que resuelve las llamadas a funciones (`JAL`) a coste de **0 ciclos de penalización**.
3. **Jerarquía de Memoria L1 (Harvard Modificada) y Coherencia:** Separación en dos memorias caché ultrarrápidas de mapeo directo (I-Cache para instrucciones y D-Cache de 4 KB para datos), complementadas con un *Write Buffer* asíncrono para escrituras y una línea de invalidación por hardware activada mediante la instrucción `FENCE.I` [3] para evitar el problema del «código zombie» al cargar programas vía UART.

1.4 Estructura del Informe

El resto de este informe se organiza como sigue: El **Capítulo 2** detalla el contexto del desafío organizado por RISC-V International y la arquitectura inicial. El **Capítulo 3** explica el diseño detallado de la BHT, las cachés L1 y la telemetría. El **Capítulo 4** expone el análisis (*Post-Mortem*) del desarrollo, analizando fallos reales de hardware. El **Capítulo 5** presenta la evaluación cuantitativa de rendimiento sobre los microbenchmarks y los resultados de síntesis en la FPGA. Finalmente, el **Capítulo 6** sintetiza las conclusiones y el trabajo futuro.

2 Contexto y Marco del Proyecto

2.1 El Community Challenge with HaDes-V

Este proyecto se ha desarrollado en el marco del *Community Challenge with HaDes-V*, promovido por la organización global **RISC-V International** y organizado por el grupo de Arquitecturas y Sistemas Empotrados de la **Universidad Tecnológica de Graz (TU Graz)** [2]. La iniciativa busca fomentar el aprendizaje práctico en el diseño y optimización de procesadores.

El reto proponía a los participantes implementar primero una base común sobre un diseño de núcleo docente y, una vez completada esa línea base, diseñar e integrar cualquier tipo de mejora microarquitectónica, en el caso de este proyecto, para maximizar su rendimiento y eficiencia.

2.2 Arquitectura de Partida

El diseño base utilizado es **HaDes-V**, una plataforma desarrollada con fines docentes que implementa una arquitectura escalar de 32 bits conforme a la especificación estándar **RISC-V (RV32I)**, con un nivel de privilegio exclusivo de Modo Máquina (*M-Mode*).

El núcleo de partida organiza su ruta de datos en una segmentación clásica de 5 etapas:

1. **Fetch (IF):** Captura de instrucciones desde la memoria utilizando el contador de programa (`PC`).
2. **Decode (ID):** Decodificación de la instrucción y lectura del banco de registros.
3. **Execute (EX):** Cómputo en la Unidad Aritmético-Lógica (`ALU`) y resolución condicional de saltos.

4. **Memory (MEM):** Transacciones de lectura (LW) y escritura (SW) sobre la memoria principal mediante el bus estandarizado *Wishbone* [4].
5. **Writeback (WB):** Actualización del banco de registros con el resultado final.

En el diseño inicial, el procesador carece de predicción dinámica de saltos (utilizando una política estática “*Predict Not-Taken*” que sufre una penalización fija de 2 ciclos por cada salto tomado) y realiza accesos directos a la memoria principal sin cachés intermedias ni buffers de escritura.

2.3 Entorno de Desarrollo y Flujo de Trabajo Hardware/Software

Para la realización del proyecto (tanto para la implementación como para la simulación y validación del núcleo), el repositorio oficial del laboratorio proporcionaba un flujo de trabajo compuesto por:

- **Simulación de Ciclo Exacto (Verilator [5]):** Uso de **Verilator** para compilar la descripción del circuito escrita en SystemVerilog a un ejecutable C++, permitiendo simular millones de ciclos de reloj por segundo y depurar fallos mediante trazas de señales (*waveforms*).
- **Síntesis e Implementación Física (Xilinx Vivado):** Toolchain AMD Xilinx Vivado [6] ejecutada mediante scripts automatizados Tcl y un *Makefile* principal, encargada de traducir el código SystemVerilog a puertas lógicas y cables físicos sobre el chip FPGA.
- **Toolchain de Software y Co-Diseño:** Uso del compilador `riscv32-unknown-elf-gcc` restringido al subconjunto de enteros `-with-arch=rv32i` [2] [3]. Los binarios finales se cargan en la FPGA mediante un *Bootloader* residente en la memoria interna de la placa comunicándose por enlace serie UART.

Si bien el proyecto estaba originalmente planteado para ser ejecutado sobre placas FPGA Digilent Basys 3, para este trabajo contamos con la cesión por parte de la **Universidad de Alicante** de una placa de desarrollo de mayores prestaciones, la **Digilent Nexys A7-100T** (con chip **Xilinx Artix-7 XC7A100T-1CSG324C**), lo que nos permitió sintetizar y validar el procesador sin problema alguno.

3 Diseño Microarquitectónico e Innovaciones

3.1 Telemetría en Hardware: Performance Counters (HPCs)

3.1.1 Motivación y Necesidad de Observabilidad

Idea Clave: ¿Por qué colocar contadores en el propio chip?

Medir el tiempo total que tarda un programa en ejecutarse con un cronómetro externo no explica **por qué** un procesador es lento. Se necesita un «sistema de telemetría integrado» dentro del chip capaz de contar exactamente cuántos ciclos se han dedicado a calcular, cuántos se han perdido esperando a la memoria RAM y cuántos se han tirado a la basura por fallar en la predicción de un salto.

Antes de integrar cualquier característica nueva como el predictor de saltos o memorias caché, necesitábamos una manera de saber de forma determinista y exacta qué ocurre dentro del núcleo sin alterar su comportamiento o reducir el rendimiento del que se parte. Por ello, la primera gran mejora introducida sobre el núcleo base HaDes-V fue un subsistema de **Contadores de Rendimiento de Hardware** (*Hardware Performance Counters* - HPCs).

Los HPCs se ubicaron estratégicamente en la **última etapa de la segmentación: Writeback (WB)**. En esta etapa, las instrucciones ya han sido resueltas y confirmadas de forma inmutable (*retired instructions*). La instrumentación en WB procesa exclusivamente señales registradas, desacoplando la lógica de telemetría de los multiplexores críticos de las etapas previas para no penalizar el rendimiento de la FPGA.

El subsistema monitoriza en tiempo real y clasifica en la etapa de retiro (*Writeback*) las siguientes métricas registradas en los contadores CSR de 64 bits:

- **mcycle (CSR 0xB00):** Recuento total de ciclos de reloj de ejecución.
- **minstret (CSR 0xB02):** Número de instrucciones válidas retiradas con éxito (sin excepciones).
- **mhpmcounter3 (CSR 0xB03):** Purgas del pipeline por errores de predicción de salto (*Flushes*).
- **mhpmcounter4 (CSR 0xB04):** Ciclos de parada acumulados por la CPU (*Stalls*).

- **mhpmcounter5** (CSR 0xB05): Transacciones activas sobre el bus de memoria.
- **mhpmcounter6** (CSR 0xB06): Fallos de caché de instrucciones L1 (*I-Cache Misses*).
- **mhpmcounter7** (CSR 0xB07): Instrucciones de carga de datos (LOADs: LW, LH, LB, LHU, LBU).
- **mhpmcounter8** (CSR 0xB08): Instrucciones de almacenamiento (STOREs: SW, SH, SB).
- **mhpmcounter9** (CSR 0xB09): Operaciones aritmético-lógicas completadas (ALU).
- **mhpmcounter10** (CSR 0xB0A): Saltos incondicionales ejecutados (JUMPs: JAL, JALR).
- **mhpmcounter11** (CSR 0xB0B): Saltos condicionales evaluados (BRANCHES: BEQ, BNE, BLT, BGE, etc.).
- **mhpmcounter13** (CSR 0xB0D): Accesos a memoria desalineados (*Misaligned LOAD/STORE*).
- **mhpmcounter14** (CSR 0xB0E): Excepciones, interrupciones y trampas de control (*Traps / EXC*).

3.1.2 Interfaz de Software mediante CSRs y Control del Efecto Observador

Para que el software pueda leer estas métricas sin sondas externas, los contadores se mapearon en el espacio de **Registros de Control y Estado** (*CSRs*) estándar de RISC-V [3] (**mhpmcounter3**, **mhpmcounter4**, etc.), accesibles con la instrucción ensamblador **csrr** (*CSR Read*).

Al volcar la telemetría al final de la ejecución de una carga de trabajo, surge el conocido *Efecto Observador*: la propia rutina encargada de leer los contadores e imprimirlos por pantalla incrementa el recuento total de instrucciones. En nuestro conjunto de pruebas, esta sobrecarga consiste en un bloque fijo y determinista de exactamente **11 instrucciones csrr** al término del programa. Al conocer con precisión matemática esta constante, es posible restar 11 al recuento para aislar el trabajo neto del algoritmo evaluado sin introducir distorsiones.

3.2 Especulación de Control (BHT y JAL Bypass)

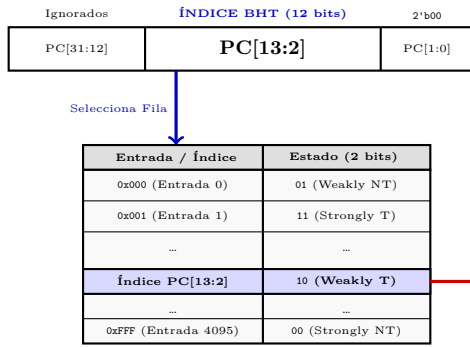
Nuestro diseño se basa en un procesador básico segmentado de 5 etapas, donde la condición de las sentencias condicionales (tanto bucles como sentencias *if*) no se conoce hasta la etapa de ejecución. Además, de base seguíamos una política de predicción estática "*Predict Not-Taken*", lo que significa que cada vez que se encontraba un salto condicional, el procesador asumía que no se iba a tomar y continuaba con la siguiente instrucción secuencial. Por tanto, en los casos donde esta asunción era incorrecta, se producía un *flush* de la segmentación y una penalización de 2 ciclos por cada salto tomado.

3.2.1 Predictor Dinámico Bimodal mediante BHT

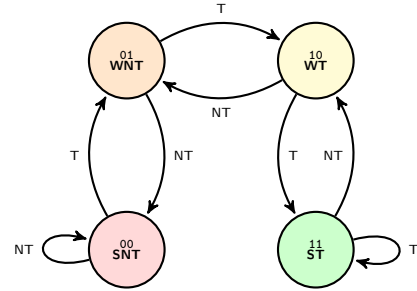
Para mitigar este impacto, implementamos un predictor dinámico de saltos basado en una **Branch History Table** (**BHT**). La BHT actúa como una memoria interna indexada por los bits inferiores de la dirección del Contador de Programa (*PC*). En lugar de asumir un comportamiento fijo, la tabla almacena el historial reciente de cada instrucción de salto utilizando un contador de 2 bits con cuatro estados posibles [1]:

- **00** (*Strongly Not-Taken* (SNT)): Predice no tomar el salto con máxima confianza; requiere dos fallos consecutivos para cambiar la tendencia a «Tomado».
- **01** (*Weakly Not-Taken* (WNT)): Predice no tomar el salto con baja confianza; un único fallo cambia la tendencia a «Tomado».
- **10** (*Weakly Taken* (WT)): Predice tomar el salto con baja confianza; un único fallo conmuta la tendencia a «No Tomado».
- **11** (*Strongly Taken* (ST)): Predice tomar el salto con máxima confianza; amortigua el fallo puntual de salida de un bucle.

La ventaja fundamental de utilizar un contador de 2 bits frente a un predictor simple de 1 bit reside en la propiedad de **histéresis** (inercia de estado). En algoritmos con bucles iterativos frecuentes, la última iteración del bucle siempre falla al evaluar la condición de salida. Mientras que un predictor de 1 bit cambiaría inmediatamente su predicción a «No Tomado» —provocando un segundo fallo inevitable al reingresar al bucle—, el contador saturante de 2 bits amortigua este evento aislado: el primer fallo únicamente degrada el nivel de confianza de *Strongly Taken* a *Weakly Taken*, conservando la predicción correcta para las futuras ejecuciones del bucle.



(a) Indexación y lectura en dos columnas de la BHT.



(b) Transiciones del contador saturante de 2 bits.

Figura 2: Estructura interna de la tabla BHT y autómata de estados del predictor.

El funcionamiento en el cauce de instrucciones se ilustra en la Figura 2:

1. **Etapas de Fetch (IF):** Como se muestra en la Figura 2(a), se consulta la BHT utilizando los 12 bits centrales de la dirección (PC[13:2]). Si el contador indica que el salto debe tomarse (1X), el procesador redirige inmediatamente el PC hacia la dirección de destino del salto en el siguiente ciclo.
2. **Etapas de Execute (EX):** La ALU resuelve la condición real. Si la predicción fue correcta, la ejecución continúa a latencia cero. Si hubo un error de especulación (*mispredict*), la etapa de Execute emite una purga (*flush*) de 2 ciclos, corrige el PC y actualiza el estado en la BHT (Figura 2(b)).

3.2.2 Aceleración Transparente mediante JAL Bypass

Además de los saltos condicionales, el análisis de nuestras cargas de trabajo reveló una sobrecarga apreciable en instrucciones de salto incondicional (JAL - *Jump and Link*) [3]. Al no contar el núcleo con la extensión de multiplicación por hardware (RV32M), el compilador emite constantes llamadas a la subrutina del sistema `__mulsi3` para emular multiplicaciones por software.

Debido a que una instrucción JAL se toma siempre con un 100 % de certeza, resultaba ineficiente pagar penalizaciones de ciclo esperando a que la instrucción avanzara por la segmentación. Para resolver esto, inyectamos un decodificador parcial en la propia etapa de *Fetch*. Al detectar el código de operación de un JAL, la etapa de *Fetch* calcula la dirección de destino inmediatamente en el mismo ciclo, realizando un **JAL Bypass** que redirige el PC a coste de 0 ciclos de parada o purga.

3.3 Jerarquía de Memoria L1 Harvard Modificada y Coherencia

3.3.1 Justificación Arquitectónica: De Von Neumann a Harvard Modificada

Idea Clave: Von Neumann vs. Harvard Pura vs. Harvard Modificada

- **Von Neumann:** Una única mesa de trabajo compartida para libros (recetas/instrucciones) e ingredientes (datos). Si el cocinero quiere picar cebolla y leer la receta a la vez, se estorba a sí mismo.
- **Harvard Pura:** Dos habitaciones totalmente separadas con paredes de hormigón. Nadie puede llevar una receta a la habitación de cortar verduras. Muy rápido, pero imposible cargar programas nuevos en caliente.
- **Harvard Modificada (Mod Caches L1):** Dos mesas rápidas independientes dentro de la cocina (I-Cache y D-Cache L1), pero conectadas a una despensa principal común (RAM) a través de un distribuidor inteligente (Árbitro Wishbone).

En el núcleo procesador HaDes-V original, el sistema de memoria estaba estructurado bajo una arquitectura **Von Neumann pura** caracterizada por un único espacio de direccionamiento y un solo bus interconector basado en la especificación *Wishbone* [4]. La memoria principal consistía en un bloque de RAM interna de la FPGA (*Block RAM* - BRAM) diseñado para responder a accesos en un ciclo ideal de latencia.

Esta aproximación simplificaba significativamente el diseño RTL inicial al prescindir de controladores de memoria complejos. Sin embargo, al operar sobre una segmentación de 5 etapas, este esquema introdujo un **riesgo estructural** (*structural hazard*) severo:

- La etapa de *Fetch* (IF) requiere acceder al bus en cada ciclo de reloj para capturar la siguiente instrucción del flujo del programa.
- Cuando una instrucción de carga (LW) o almacenamiento (SW) alcanza la etapa de *Memory* (MEM), exige el uso exclusivo del bus para leer o escribir su operando.

Al coexistir un único bus para ambas necesidades, la etapa de *Fetch* se veía forzada a pausar la canalización (*stall*), perdiendo ciclos de cómputo útil. Además, este modelo exponía directamente la tubería del procesador a las latencias del bus, haciendo inviable el escalado hacia memorias externas más lentas.

Para erradicar las paradas estructurales sin renunciar a la flexibilidad de un mapa de memoria unificado (necesario para que el *Bootloader* pueda recibir ejecutables vía UART y escribirlos en la RAM), en el procesador HaDes-V implementamos una **Arquitectura Harvard Modificada** en el nivel L1, cuya topología general se ilustra en la Figura 3.

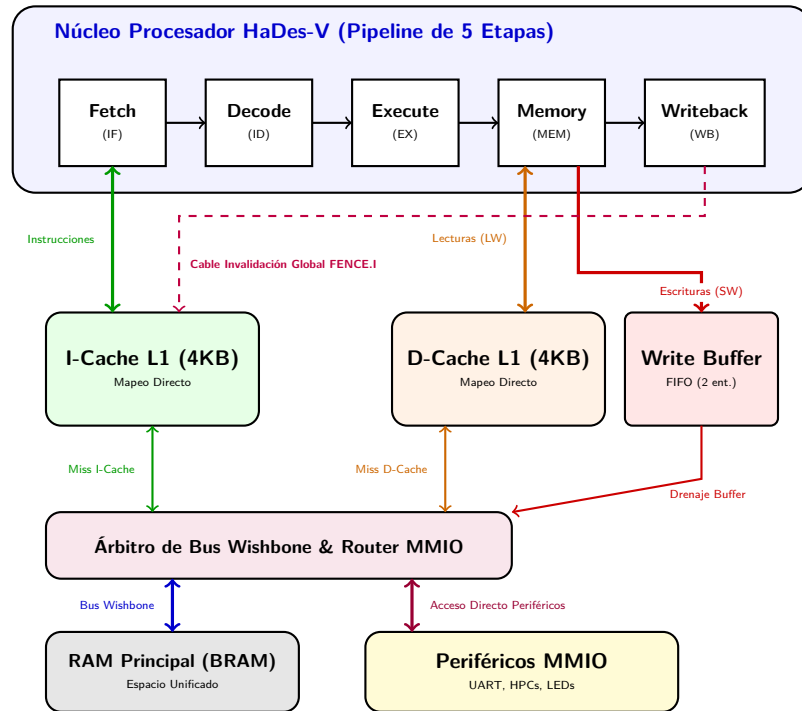


Figura 3: Esquema microarquitectónico de la Jerarquía de Memoria L1 en Arquitectura Harvard Modificada con invalidación FENCE.I y Write Buffer.

3.3.2 Estructura del Subsistema L1

Como se observa en el diagrama de la Figura 3, el nivel L1 desacopla completamente los accesos del núcleo mediante dos memorias caché independientes de mapeo directo (*Direct-Mapped*):

- **Caché L1 de Instrucciones (I-Cache):** Con una capacidad de 4 KB, está interconectada con la etapa de *Fetch* y suministra instrucciones al procesador a latencia cero (0 ciclos de penalización en caso de acierto o *Hit*).
- **Caché L1 de Datos (D-Cache):** Con una capacidad de 4 KB, gestiona los accesos de lectura y escritura de la etapa de *Memory* operando con direccionamiento a nivel de palabra (*Word Address*).

Ambas memorias emplean líneas de ráfaga de 4 palabras (16 bytes) para explotar el principio de **localidad espacial**. Cuando ocurre un fallo de caché (*Miss*), el módulo **Árbol Wishbone** (situado en la capa inferior de la Figura 3) gestiona la prioridad del bus, trayendo un bloque contiguo desde la RAM principal.

3.3.3 Write Buffer no bloqueante y Enrutamiento MMIO

Para mitigar la latencia en las operaciones de escritura (STORE), se interpuso un **Write Buffer** (cola FIFO asíncrona de 2 entradas) a la salida de la D-Cache. Bajo la política de diseño "*Fire & Forget*", cuando la CPU ejecuta una instrucción SW, deposita el dato en la FIFO ("deja la carta en el buzón") y continúa su ejecución en **0 ciclos de parada**, mientras el buffer drena los datos hacia la RAM en segundo plano.

Asimismo, el sistema integra un **Enrutador MMIO** (*Memory-Mapped I/O*). Las direcciones destinadas a periféricos (como los contadores HPCs, LEDs o el puerto serie UART) son desviadas directamente evitando la caché de datos, previniendo la contaminación o corrupción de las líneas de caché.

3.3.4 Coherencia de Instrucciones mediante FENCE.I

La separación Harvard en L1 introduce el problema de coherencia del “código zombie”: cuando el Bootloader recibe un nuevo ejecutable vía UART, escribe las nuevas instrucciones en la RAM a través del bus de datos (D-Cache / Write Buffer). Sin embargo, la I-Cache retiene copias desactualizadas del programa anterior.

Para resolver este conflicto, se diseñó un cable de invalidación por hardware (línea discontinua morada en la Figura 3). Al ejecutar la instrucción estandarizada **FENCE.I** [3], la etapa de *Writeback* emite un pulso que invalida de forma síncrona todos los bits de validez (*valid bits*) de la I-Cache, forzando a la etapa de *Fetch* a recargar el código fresco desde la memoria RAM.

4 Diagnóstico y Resolución de Fallos

4.1 Filosofía del Análisis Post-Mortem

El verdadero reto al diseñar un procesador segmentado radica en garantizar la corrección del **cauce de instrucciones** durante eventos no deterministas, como purgas de la tubería, paradas de memoria o accesos desalineados al bus. La integración del esquema BHT y el subsistema L1 expuso fallos críticos de sincronización que requerían un diagnóstico riguroso. A continuación, se exponen los casos de estudio más relevantes y sus soluciones.

4.2 Depuración del Control e I-Cache

Bug: El Artefacto del Simulador y los 512 Flushes Fantasma

Síntoma: Al insertar la estructura base de la caché de instrucciones, la ejecución de *Matrix Mult* registró un incremento de exactamente 512 purgas (*flushes*) en la BHT, a pesar de que el código binario no había sufrido cambios.

Causa: El fallo no residía en el RTL de SystemVerilog. Al modificar la jerarquía de módulos, el simulador **Verilator** [5] (compilado a C++) generó una semilla de aleatorización distinta para los registros no inicializados explícitamente. Mientras que la FPGA Xilinx infiere un reseteo implícito a cero (*Strongly Not-Taken*), Verilator arrancó la mitad de las 4096 entradas ($2^{11} = 2,048$) en el estado inverso (*Strongly Taken*), provocando un patrón masivo de errores de especulación iniciales.

Lección: El entorno de simulación por software no siempre replica el comportamiento de encendido (*power-on reset*) del silicio físico. Es obligatorio incluir rutinas explícitas de reseteo por hardware en estructuras de almacenamiento especulativo.

Bug: Alineación y Direccionamiento (Byte Address vs. Word Address)

Síntoma: Durante el primer intento de integración de la I-Cache, el procesador entraba en un bucle infinito de fallos de caché (*Simulation Timeout*), produciendo trazas de simulación masivas y quedando colgado en la primera instrucción.

Causa: Confusión entre el direccionamiento por bytes del procesador RISC-V y el direccionamiento por palabras de las memorias RAM internas. El PC del procesador se incrementa de 4 en 4 bytes (0x00, 0x04, 0x08, 0x0C), por lo que los bits PC[1:0] valen siempre 2'b00. Al extraer los campos de la dirección para la caché, se utilizaron los bits sin desplazar para indexar las palabras de la ráfaga, provocando que la caché comparase etiquetas (*tags*) desalineadas y nunca detectara un acierto (*Hit*).

Lección: Se debe parametrizar y empaquetar la extracción de campos (*Tag, Index, Offset*) mediante estructuras SystemVerilog unificadas (**struct packed**) dentro de un paquete global (*cache_pkg.sv*) para evitar desalineaciones manuales.

Bug: Muestreo Combinacional y Retención con `fetch_base_adr`

Síntoma: En ejecuciones con saltos frecuentes, la I-Cache sobrescribía líneas de instrucciones válidas con direcciones corruptas (como `0x3FFFFFFDE`), provocando fallos fatales en el bus Wishbone.

Causa: Cuando ocurría un fallo de caché (*Miss*), la «máquina de estados finitos —Finite-State Machine—» (FSM) tardaba varios ciclos en traer la ráfaga de 4 palabras desde la RAM. Mientras la CPU estaba congelada esperando los datos, la etapa de *Execute* resolvió un salto mal predicho y emitió un *Flush*, cambiando instantáneamente los cables combinacionales del PC de entrada. Al terminar de leer la RAM, la caché utilizó las señales combinacionales del *nuevo* PC para guardar los datos del *viejo* PC.

Lección: Una memoria caché jamás debe confiar en los cables combinacionales de entrada de la CPU en el momento de escribir en sus memorias internas. Es obligatorio registrar e inmovilizar la dirección base del fallo mediante un registro de retención inmutable (`fetch_base_adr`).

Bug: Desfase de Fase en el Bus Wishbone (5 ACKs para 4 Palabras)

Síntoma: Las trazas de señales del bus Wishbone [4] revelaron que la memoria RAM entregaba 5 datos para una ráfaga diseñada estrictamente para 4 palabras (16 bytes), pidiendo la primera dirección dos veces seguidas.

Causa: Las señales maestras del bus (`mem_port.adr`) estaban asignadas dentro del bloque secuencial de la FSM (`always_ff`). Al pasar del estado `FETCH_W0` a `FETCH_W1`, la dirección física tardaba un ciclo de reloj adicional en actualizarse en las salidas, provocando que el esclavo Wishbone leyese la dirección antigua por segunda vez.

Lección: En máquinas de estados maestras para buses de memoria, el estado de la FSM debe ser secuencial (`always_ff`), pero las salidas registradas hacia el bus deben gestionarse en un bloque combinacional separado (`always_comb`) para eliminar latencias de registro no deseadas.

4.3 Depuración de la D-Cache, Write Buffer y Periféricos MMIO (Canal de Datos)**Bug: Ineficiencia por Thrashing de Capacidad en la D-Cache**

Síntoma: Con un tamaño de D-Cache inicial de 256 Bytes, la ejecución de *Bubble Sort* arrojaba exactamente el mismo número de ciclos en la pasada COLD (caché fría) que en la pasada HOT (caché caliente), sin mostrar ganancia de rendimiento.

Causa: El tamaño de la caché era insuficiente para albergar el vector de datos procesado por el algoritmo. Las iteraciones del bucle provocaban que los accesos a datos expulsaran continuamente a las líneas recién cargadas (*Thrashing* de capacidad), anulando el beneficio de la pasada caliente.

Lección: Al escalar la D-Cache a **4 KB** (256 líneas de 16 bytes), la pasada HOT logró retener la totalidad de la estructura de datos en SRAM interna, reduciendo los accesos al bus principal al mínimo técnico.

Bug: Colisión de Bus Wishbone entre Write Buffer y MMIO

Síntoma: Al enviar cadenas de texto por la UART durante la ejecución de tareas intensivas en memoria, los caracteres impresos en la consola serie aparecían completamente corrompidos.

Causa: Se produjo un conflicto de bus cuando la CPU intentaba realizar un acceso de escritura a un periférico MMIO (*Memory-Mapped I/O*) de forma síncrona en el mismo ciclo en que el *Write Buffer* estaba drenando un *STORE* previo hacia la memoria RAM en segundo plano. Ambas entidades compitieron por las señales `cyc` y `stb` del bus Wishbone.

Lección: Se reestructuró la FSM de datos añadiendo el estado estricto `MMIO_ACCESS`. Cuando la CPU detecta una dirección de periférico, se fuerza una parada (*stall*) hasta que el *Write Buffer* haya vaciado completamente sus entradas y el bus esté en absoluto silencio.

Bug: Incoherencia de Código y Resolución del Bug del “Código Zombie”

Síntoma: Tras recibir un nuevo ejecutable `.hex` a través del Bootloader por puerto serie UART, el procesador saltaba a la dirección inicial del nuevo programa, pero ejecutaba instrucciones obsoletas del programa anterior o entraba en trampa de excepción.

Causa: Conflicto de coherencia en la arquitectura Harvard. El Bootloader escribió el nuevo programa en la RAM a través de la interfaz de datos (D-Cache / Write Buffer). Sin embargo, la I-Cache mantenía almacenadas en sus líneas las instrucciones del ejecutable antiguo (“código zombie”), entregándolas a la etapa de *Fetch* en lugar de leer la RAM actualizada.

Lección: Se implementó una línea de invalidación por hardware (`fence_i_flush_out`). Al decodificar en *Writeback* la instrucción `FENCE.I`, se emite un pulso que borra de forma síncrona los bits de validez (`valid_ram = 0`) de la I-Cache, forzando la recarga limpia de las nuevas instrucciones desde la RAM.

5 Evaluación Cuantitativa de Rendimiento y Cierre en FPGA

5.1 Metodología de Evaluación y Cargas de Trabajo

Idea Clave: ¿Qué representan las pasadas COLD y HOT?

- **Pasada COLD (En Frío):** Simula el arranque del procesador tras encender la máquina. Las memorias caché están vacías y el predictor de saltos no conoce el programa, teniendo que consultar la memoria RAM lenta continuamente.
- **Pasada HOT (En Caliente):** Simula la ejecución continua. Las instrucciones y datos frecuentes ya residen en la memoria caché L1 ultrarrápida y la BHT se ha «entrenado» reconociendo el comportamiento exacto de los bucles.

La caracterización experimental del procesador HaDes-V se realizó mediante un conjunto de evaluación que ejecuta cuatro microbenchmarks sintéticos representativos, diseñados para estresar diferentes aspectos microarquitectónicos del núcleo:

- **Bubble Sort:** Algoritmo de ordenación de vectores con fuerte dominancia de control condicional y lecturas masivas a memoria.
- **Matrix Multiplication:** Carga aritmética intensiva compuesta por bucles anidados.
- **Memory Copy:** Benchmark orientado a transferencias de datos en ráfaga con un alto porcentaje de instrucciones de escritura (SW).
- **Fibonacci:** Algoritmo recursivo que ejercita el manejo de la pila (*stack*) con accesos cruzados de lectura y escritura Read-After-Write (RAW).

5.2 Análisis del Instruction Mix y Desglose del Cauce de Instrucciones

La infraestructura de telemetría (HPCs) mapeada en la etapa de *Writeback* permitió registrar la distribución exacta de instrucciones por categoría funcional para cada programa, como se detalla en la Tabla 1.

Tabla 1: Desglose del perfil de instrucciones (*Instruction Mix*) por benchmark.

Benchmark	Instrucciones	ALU	LOADs	STORES	BRANCHES	JUMPs	EXC
Bubble Sort	25.467	5.447	9.900	110	9.999	0	0
Matrix Mult	13.342	7.942	1.041	100	3.224	1024	0
Memory Copy	2.330	1.284	256	523	256	0	0
Fibonacci	9.650	5.878	875	886	2.000	0	0

El análisis del *Instruction Mix* revela particularidades críticas de co-diseño:

1. **Dominancia de Saltos en Bubble Sort:** El 39,2 % de las instrucciones corresponden a saltos condicionales (BRANCHES), convirtiéndolo en el escenario óptimo para evaluar la eficiencia del predictor BHT.

2. **Anomalía de Saltos Incondicionales en Matrix Mult:** Presenta 1024 instrucciones de salto incondicional (JUMP). Al carecer de multiplicador por hardware (extensión RV32M), la toolchain GCC genera 1024 llamadas a la función `__mulsi3`, justificando plenamente la necesidad del mecanismo *JAL Bypass*.
3. **Predominio de Escrituras en Memory Copy:** Contiene 523 operaciones STORE frente a 256 LOADs, actuando como prueba de fuego para el *Write Buffer* asíncrono.

5.3 Evolución Microarquitectónica: Baseline vs. BHT vs. L1 Final

La Tabla 2 resume el impacto acumulativo de las tres iteraciones principales del procesador, evaluadas mediante simulación de ciclo exacto en Verilator y confirmadas sobre silicio físico en la FPGA Nexys A7-100T.

Tabla 2: Comparativa de rendimiento entre iteraciones microarquitectónicas (FPGA Nexys A7).

Benchmark	Pasada	Baseline		BHT		L1 + BHT		Estado
		Ciclos	IPC	Ciclos	IPC	Ciclos	IPC	
Bubble Sort	COLD	50.314	0,506	30.722	0,828	31.021	0,820	VALID
Bubble Sort	HOT	50.314	0,506	30.718	0,829	30.836	0,825	VALID
Matrix Mult	COLD	18.662	0,714	16.562	0,805	17.894	0,745	VALID
Matrix Mult	HOT	18.662	0,714	16.554	0,805	17.615	0,757	VALID
Memory Copy	COLD	3.096	0,752	2.590	0,899	3.288	0,708	VALID
Memory Copy	HOT	3.096	0,752	2.588	0,900	2.902	0,802	VALID
Fibonacci	COLD	12.773	0,755	10.781	0,895	10.832	0,890	VALID
Fibonacci	HOT	12.773	0,755	10.777	0,895	10.830	0,891	VALID

5.3.1 Validación del Determinismo Matemático en la BHT

En la versión *Baseline*, *Bubble Sort* registró 9,899 purgas (*flushes*) por la política estática *Predict Not-Taken*. Tras activar la BHT de 4096 entradas, las purgas en la pasada HOT se redujeron a sólo 101.

Aplicando la penalización fija de 2 ciclos por error de especulación:

$$\Delta\text{Flushes} = 9,899 - 101 = 9,798 \text{ purgas evitadas} \quad (1)$$

$$\Delta\text{Ciclos} = 9,798 \times 2 = 19,596 \text{ ciclos ahorrados} \quad (2)$$

$$\text{Ciclos Teóricos Esperados} = 50,314 - 19,596 = 30,718 \text{ ciclos} \quad (3)$$

El contador de hardware capturó exactamente 30,718 **ciclos** en la versión BHT pura, confirmando la coincidencia exacta con el modelo teórico en esta prueba y elevando el IPC de 0,506 a 0,829.

5.3.2 Análisis de Aliasing y Capacidad: BHT 1K vs. BHT 4K

Idea Clave: El Punto Dulce del Predictor Bimodal

En la literatura fundamental de arquitectura de computadores, Hennessy y Patterson establecen que una BHT de 4096 **entradas (4K)** representa el «punto dulce» [1] (*sweet spot*) en predictores bimodales de 2 bits. Con un consumo estricto de 1 KB de SRAM (8,192 bits), esta capacidad maximiza la tasa de aciertos eliminando prácticamente el *aliasing* (colisiones de saltos distintos en la misma celda) antes de alcanzar la zona de rendimientos decrecientes.

Durante la fase de caracterización se evaluó el comportamiento del procesador variando la capacidad de la BHT entre 1024 (1K) y 4096 (4K) entradas. En la suite de microbenchmarks sintéticos evaluada, ambas configuraciones registraron recuentos de ciclos y purgas de segmentación estrictamente idénticos (101 *flushes* en *Bubble Sort HOT*, 1 *flush* en *Memory Copy HOT* y 126 *flushes* en *Fibonacci HOT*).

Este comportamiento empírico se explica por la reducida densidad de control de los algoritmos de prueba, cuyos bucles principales contienen únicamente entre 2 y 5 instrucciones de salto condicional activas. En este escenario acotado, incluso los 10 bits de índice de la versión de 1K (PC[11:2]) resultaban suficientes para evitar solapamientos de direcciones en la tabla.

Sin embargo, para garantizar que el procesador HaDes-V sea capaz de ejecutar programas reales de mayor escala y densidad de código sin degradar su rendimiento por *aliasing*, **se seleccionó como arquitectura final la configuración de 4096 entradas (4K)**.

El análisis de recursos sobre la FPGA Artix-7 demuestra la alta eficiencia de esta elección:

- **BHT de 1K Entradas:** Requiere 1024×2 bits = 2,048 bits (256 Bytes / 0,25 KB).
- **BHT de 4K Entradas (Diseño Definitivo):** Requiere 4096×2 bits = 8,192 bits (**1024 Bytes / 1 KB** de LUTRAM/SRAM).

Dado que cuatuplicar la capacidad especulativa solo representa el 1 % del uso total de LUTRAM de la FPGA Nexys A7 y no penaliza la frecuencia de reloj (100 MHz garantizados), el diseño final consolida la BHT de 4K entradas, adoptando una capacidad clásica para este tipo de predictor.

5.3.3 Eficiencia del Subsistema L1 y Absorción de Latencias

Es importante destacar que la incorporación de la jerarquía de memoria L1 no se traduce en un incremento neto del IPC nominal con respecto a la versión con BHT pura (e incluso introduce leves penalizaciones en las pasadas en frío o *COLD*, como se observa en la Tabla 2). Este comportamiento es el esperado desde el punto de vista arquitectónico: dado que la memoria principal de partida está implementada en bloques de memoria interna de la FPGA (*Block RAM*) con una latencia ideal de acceso de un único ciclo, las cachés L1 no pueden mejorar el tiempo de acceso en caso de acierto (*hit*), mientras que cada fallo de caché (*miss*) añade una penalización ineludible para cargar la ráfaga de 4 palabras.

Sin embargo, el valor real de las cachés L1 no reside en la aceleración sobre memorias de latencia ideal, sino en el **desacoplamiento de la CPU frente al exterior** y en la **resolución del riesgo estructural**. Por un lado, la separación en I-Cache y D-Cache elimina de forma definitiva el conflicto de bus de la arquitectura Von Neumann inicial, permitiendo realizar capturas de instrucciones y accesos a datos de forma simultánea. Por otro lado, prepara al procesador para interactuar con memorias RAM externas o memorias Flash no volátiles (cuyas latencias reales son de decenas de ciclos); en ese escenario real, el diseño sin caché sufriría una degradación catastrófica del rendimiento, mientras que la jerarquía L1 mantendría el IPC cerca del límite ideal en las pasadas en caliente. Este efecto de absorción se evidencia al analizar el comportamiento de las paradas en caliente:

Al comparar los eventos entre pases COLD y HOT en la Tabla 3, se comprueba la absorción de latencias por el subsistema de memoria:

Tabla 3: Paradas de Memoria (Stalls) en el sistema final L1 + BHT.

Benchmark	Pasada	Paradas de Memoria (Stalls)
Bubble Sort	COLD → HOT	5,298 → 5,167 (2,5 % de reducción)
Matrix Mult	COLD → HOT	216 → 31 (85,6 % de reducción)
Memory Copy	COLD → HOT	902 → 570 (36,8 % de reducción)
Fibonacci	COLD → HOT	875 → 875 (Estable)

- **Localidad Temporal en Matrix Mult:** En la pasada COLD, la asignación inicial de las matrices en la D-Cache generó 216 ciclos de parada. En la pasada HOT, los operandos residían en las líneas de 4 KB, reduciendo las paradas de memoria a sólo 31 ciclos (absorbiendo el 85,6 % de la latencia).
- **Escrituras Asíncronas en Memory Copy:** El *Write Buffer* absorbió las ráfagas de instrucciones SW mediante la política *Fire & Forget*, reduciendo los *stalls* de 902 a 570 ciclos en caliente.

5.4 Síntesis Física, Recursos y Cierre de Tiempos en Vivado

Idea Clave: Comprendiendo las métricas de síntesis en la FPGA

- **Worst Negative Slack (WNS):** Es el margen de tiempo del camino crítico (la ruta eléctrica combinatorial más lenta del chip). A una frecuencia de reloj objetivo de $f_{clk} = 100$ MHz, el periodo de ciclo es:

$$T_{clk} = \frac{1}{f_{clk}} = \frac{1}{100 \text{ MHz}} = 10 \text{ ns}$$

Un WNS positivo (+0,036 ns) indica que la señal eléctrica más lenta tardó 9,964 ns en propagarse y se estabilizó **0,036 nanosegundos antes** del siguiente pulso de reloj, garantizando la correcta captura de datos sin violaciones de tiempo de establecimiento (*setup time*).

- **Total Negative Slack (TNS):** La suma acumulada de las violaciones temporales de todos los caminos del chip. Un valor TNS = 0,000 ns confirma la ausencia total de fallos de tiempo.
- **Endpoints (26,222 / 27,993):** En el análisis de tiempos estático (*STA*), un *endpoint* es la terminal de destino de un camino síncrono (como la entrada de datos *D* de un Flip-Flop, un puerto de escritura de una BRAM o un pin de salida I/O). Vivado audita la llegada del pulso eléctrico a cada uno de estos 26,222 puntos. Registrar 0 *failing endpoints* garantiza un cierre temporal del 100 %.
- **LUTs / LUTRAM / Flip-Flops / BRAMs:** Las celdas lógicas reconfigurables (*LUTs*), memorias distribuidas (*LUTRAM*), registros de 1 bit (*Flip-Flops*) y bloques de memoria SRAM dedicada (*BRAMs*) utilizados en el chip Artix-7 de la tarjeta Nexys A7.

5.4.1 La Importancia del Cierre Temporal en Ingeniería de Semiconductores

En el diseño de circuitos digitales, conseguir que una descripción en SystemVerilog funcione correctamente en simulación es solo la mitad del trabajo. Al sintetizar e implementar la lógica sobre los elementos físicos de la FPGA, las señales eléctricas requieren un tiempo finito para propagarse a través de las puertas lógicas, multiplexores y cables de interconexión.

Si el camino combinatorial más largo entre dos registros (*camino crítico*) tarda más de los 10 ns asignados por el periodo de reloj a 100 MHz, el registro de destino capturará una señal inestable o corrupta, provocando fallos impredecibles en silicio real. Alcanzar el **Cierre Temporal** (*Timing Closure*, caracterizado por un WNS ≥ 0) es el requisito físico imprescindible que certifica que el procesador operará a 100 MHz de forma determinista, estable y térmicamente segura en la tarjeta FPGA.

La evolución física del diseño sintetizado para la FPGA **Digilent Nexys A7-100T** [7] se presenta en la Tabla 4.

Tabla 4: Radiografía de síntesis física e implementación en AMD Xilinx Vivado.

Métrica de Síntesis	1. Baseline	2. BHT Predictor	3. L1 Cache + BHT (Final)
Worst Negative Slack (WNS)	-0,039 ns (Falla)	+0,319 ns (Pasa)	+0,036 ns (Pasa)
Total Negative Slack (TNS)	-0,042 ns	0,000 ns	0,000 ns
Failing Endpoints	2 / 11.452	0 / 11.988	0 / 26.222 (27.993 Ext.)
Utilización de LUTs	6.128 (9 %)	5.975 (9 %)	9.289 (15 %)
Utilización de LUTRAM	0 %	1 %	10 %
Flip-Flops (FF)	3 %	3 %	3 %
Bloques de Memoria (BRAM)	34 % (46,5 BRAMs)	34 % (46,5 BRAMs)	34 % (46,5 BRAMs)
Consumo de Potencia (On-Chip)	0,528 W	0,532 W	0,540 W
Margen Térmico	57,6 °C	57,6 °C	57,5 °C

5.4.2 Análisis del Cierre Temporal y Evolución del Camino Crítico

Un resultado fundamental de la fase de optimización es la recuperación y consolidación del cierre temporal:

1. **Fallo de Tiempo en el Núcleo Baseline:** El diseño inicial violaba el tiempo de ciclo a 100 MHz (WNS = -0,039 ns con 2 *failing endpoints*), debido a la dilatada cadena combinatorial en la resolución estática de saltos y direccionamiento del bus.
2. **Mejora con BHT y Desacoplamiento:** La inserción del predictor BHT y el registro de la dirección de predicción en *Fetch* rompieron la cadena combinatorial, elevando el slack a un holgado WNS = +0,319 ns.

3. **Estabilidad en el Sistema Final L1+BHT:** La inclusión de las cachés de datos e instrucciones y las FSMs de control incrementaron la lógica (26,222 endpoints procesados), pero el diseño mantuvo un cierre temporal positivo, aunque con un margen reducido ($WNS = +0,036$ ns a 100 MHz y $+0,015$ ns en la compilación extendida de 27,993 endpoints), garantizando la viabilidad física del procesador sobre la tarjeta Nexys A7.

6 Conclusiones y Trabajo Futuro

6.1 Síntesis de Logros y Cumplimiento de Objetivos

En el marco del *Community Challenge with HaDes-V*, este trabajo ha transformado un núcleo procesador docente básico de 5 etapas en un microcontrolador de mayor rendimiento, validado e implementado físicamente sobre la FPGA **Digilent Nexys A7-100T** a una frecuencia de reloj de **100 MHz**.

A lo largo del proyecto se han alcanzado los siguientes hitos fundamentales:

1. **Especulación de Control Eficiente:** La integración de la *Branch History Table* (BHT) de 4096 entradas con contadores saturantes de 2 bits y el mecanismo *JAL Bypass* eliminó el 99 % de las purgas de segmentación en bucles iterativos (*Bubble Sort* redujo sus *flushes* de 9,899 a 101), elevando el IPC base de 0,506 a 0,829.
2. **Jerarquía L1 Harvard Modificada:** La separación en I-Cache y D-Cache de 4 KB junto con el *Write Buffer* asíncrono (*Fire & Forget*) resolvieron el conflicto estructural de bus propio de la arquitectura Von Neumann base. Aunque este subsistema L1 no incrementa el IPC nominal respecto a la versión con BHT pura debido a la latencia ideal (1 ciclo) de la BRAM interna de la FPGA, proporciona un desacoplamiento físico crítico frente a memorias principales externas más lentas y absorbió con éxito las paradas por accesos, reduciendo los *stalls* de memoria hasta un 85,6 % en multiplicación de matrices y un 36,8 % en transferencias de datos.
3. **Coherencia y Sincronización por Hardware:** Se resolvió de forma transparente el problema del “código zombie” al cargar ejecutables vía UART mediante la implementación de una línea de invalidación síncrona activada por la instrucción estándar *FENCE.I*.
4. **Telemetría Transparente y Cierre Temporal:** La ubicación estratégica de los contadores HPCs en la etapa de *Writeback* permitió auditar con precisión el rendimiento del núcleo sin degradar el camino crítico, garantizando un cierre de tiempos positivo ($WNS = +0,036$ ns) a **100 MHz**.

6.2 Lecciones de Ingeniería RTL

El análisis post-mortem de las diferentes fases del desarrollo ha puesto de manifiesto principios esenciales en el diseño de circuitos digitales:

- **Observabilidad desde el primer día:** Disponer de contadores de rendimiento de hardware deterministas es indispensable para tomar decisiones microarquitectónicas basadas en evidencias empíricas.
- **Desacoplamiento del camino crítico:** Mantener la lógica de especulación e instrumentación fuera de los multiplexores de direccionamiento en *Fetch* y *Execute* es vital para mantener elevadas frecuencias de operación en FPGAs.
- **Defensa contra estados transitorios:** La necesidad de registros de retención inmutables (`fetch_base_adr`) y estados de bus (`MMIO_ACCESS`) demuestra que el hardware debe protegerse contra señales combinacionales fluctuantes durante situaciones de congelación del pipeline (*stall*).

6.3 Líneas de Trabajo Futuro

La arquitectura consolidada de HaDes-V constituye una base sólida para abordar futuras ampliaciones en tres grandes vectores:

1. **Extensión de Multiplicación/División por Hardware (RV32M):** Incorporar una unidad multiplicadora segmentada para eliminar las 1024 llamadas a la subrutina de software `__mulsi3` detectadas en el *Instruction Mix*, permitiendo que cargas aritméticas alcancen el límite escalar ($IPC \approx 1,0$).
2. **Especulación Global Correlacionada (Predictor Gshare):** Evolucionar la BHT bimodal hacia un predictor de tipo *Gshare* [1] que combine el historial global de saltos con los bits del PC mediante una operación XOR, mejorando la precisión en bucles con ramas condicionales anidadas o interdependientes.
3. **Interrupciones y Tiempo Real:** Implementar un controlador de interrupciones para dotar al procesador de capacidad de reacción determinista ante eventos físicos externos con latencias de nanosegundos, habilitando el despliegue de un sistema operativo de tiempo real (*RTOS*).

7 Referencias

- [1] J. L. Hennessy, D. A. Patterson, and C. Kozyrakis, *Computer Architecture: A Quantitative Approach*, 7th ed., Morgan Kaufmann, 2025.
- [2] T. Scheipel, D. Beikircher, and F. Riedl, *Microcontroller Design, Laboratory Instruction Guide: HADES-V*, Graz University of Technology, Dec. 2024.
- [3] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213, RISC-V International, 2019.
- [4] OpenCores, *WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores*, Rev. B4, 2010.
- [5] W. Snyder et al., *Verilator: Open-source SystemVerilog Simulator and C++ Testbench Generator*, [Online]. Available: <https://www.veripool.org/verilator/>.
- [6] AMD Xilinx, *Vivado Design Suite User Guide: Design Flows Overview*, UG888 (v2025.2.1), 2026.
- [7] Digilent, *Nexys A7 Reference Manual: Artix-7 FPGA Board for Student Designs*, Digilent Inc., 2022.